

Room Occupancy Spectrogram Classification

Experimental Report Draft

YANG Chen

Table of contents

1	Introduction	2
2	Experimental Setup	2
3	Overall Experimental Framework	2
3.1	Data Preparation	4
3.2	Baseline Training	4
3.3	Baseline Comparison	4
3.4	Augmentation Comparison	4
3.5	Validation Analysis	4
3.6	Final Cross-validation	4
3.7	Submission Generation	4
4	Dataset Overview	5
5	Preprocessing Design	5
5.1	Three Augmentation Strategies	6
5.1.1	Strategy: none	6
5.1.2	Strategy: light	6
5.1.3	Strategy: spectral_safe	6
6	Baseline Model Comparison	7
7	Augmentation Comparison	9
8	Best Holdout Model Analysis	11
8.1	Key Holdout Metrics	11
9	Final 5-Fold Cross-Validation Evaluation	14
10	Final Submission Artifact	17
11	Conclusion	18

1 Introduction

This report summarizes the development of a spectrogram-based room occupancy classification pipeline. The work progressed from a simple baseline comparison to augmentation design, validation error analysis, and finally a full stratified 5-fold cross-validation evaluation.

The main goal of the project is to build a robust classifier for room occupancy states from spectrogram images while keeping the pipeline reproducible and the conclusions supported by multiple evaluation stages.

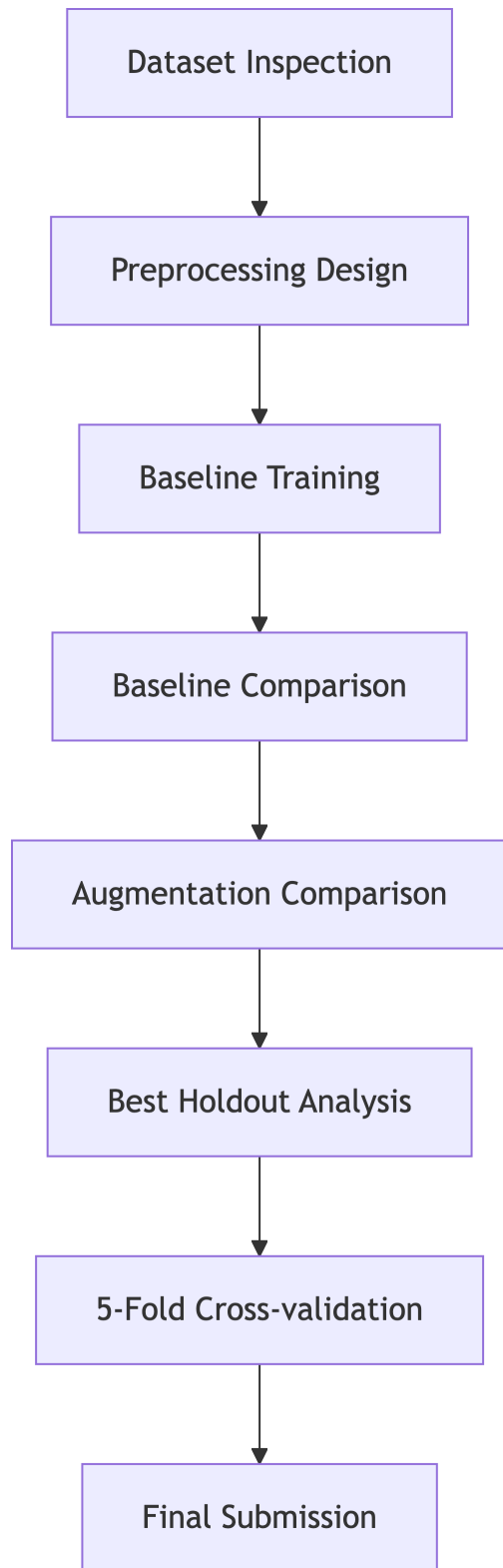
2 Experimental Setup

item		value
0	Model	resnet18
1	Augmentation	spectral_safe
2	Input Size	64 x 64
3	Input Mode	grayscale
4	Evaluation Confirmation	holdout + 5-fold

3 Overall Experimental Framework

The project was organized as a staged experimental pipeline rather than a single training script. This design was important for two reasons. First, it kept the workflow reproducible because each stage produced explicit artifacts that could be inspected later. Second, it made the project easier to extend from a simple holdout experiment to validation analysis and finally to 5-fold cross-validation.

At a high level, the workflow followed the sequence below:



The main stages of the framework are summarized below.

3.1 Data Preparation

- Goal: Load image paths and labels, inspect dataset structure, and verify class balance.
- Main artifacts: EDA tables, label distribution, sample images

3.2 Baseline Training

- Goal: Train baseline models under a unified pipeline and save reproducible artifacts.
- Main artifacts: checkpoints, training_history.csv, validation_predictions.csv

3.3 Baseline Comparison

- Goal: Compare different backbones on the same holdout split.
- Main artifacts: baseline_comparison_none.csv

3.4 Augmentation Comparison

- Goal: Evaluate augmentation presets for the strongest baseline model.
- Main artifacts: resnet18_aug_comparison.csv

3.5 Validation Analysis

- Goal: Inspect confusion patterns, class metrics, and representative errors.
- Main artifacts: confusion_matrix.csv, class_metrics.csv, misclassified_samples.csv

3.6 Final Cross-validation

- Goal: Verify that the best configuration remains stable across multiple folds.
- Main artifacts: kfold/summary.csv, kfold/fold_metrics.csv, kfold/confusion_matrix.csv

3.7 Submission Generation

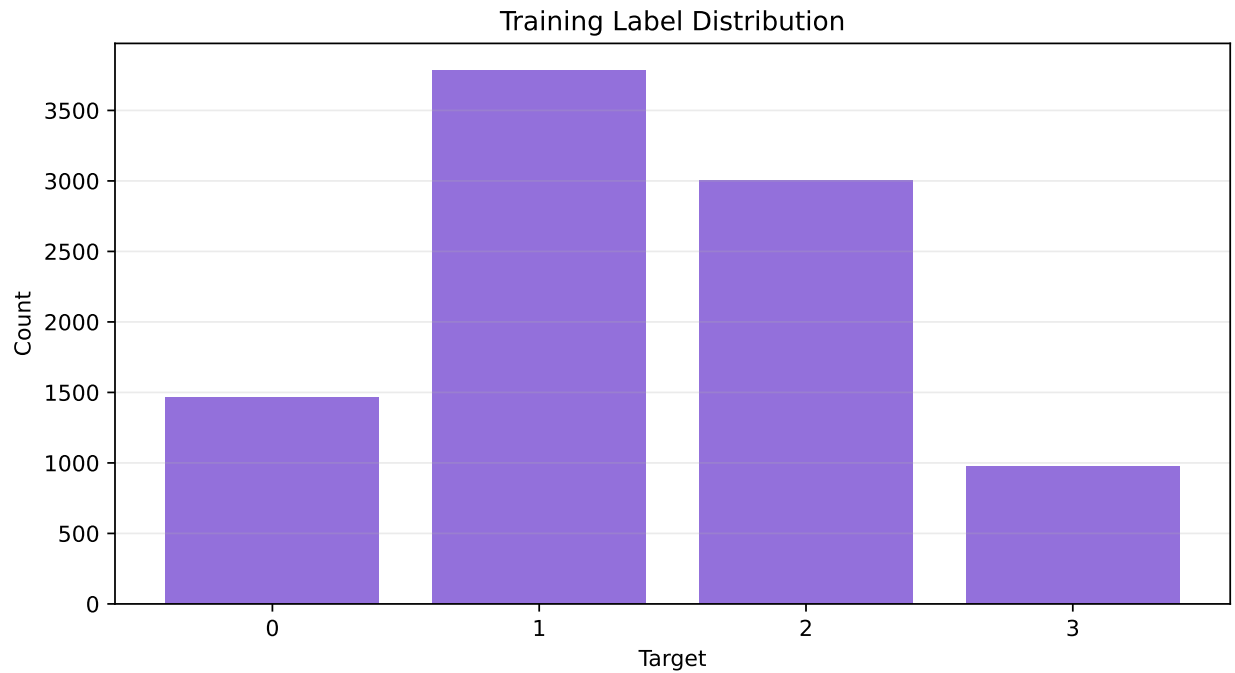
- Goal: Export a Kaggle-ready CSV from the best available checkpoint.
- Main artifacts: submission_resnet18.csv

From an engineering perspective, the framework is implemented through a unified `run.py` entry point. Different project actions are dispatched through configuration-controlled pipelines, including training, comparison, validation analysis, submission generation, and final cross-validation. This structure ensures that the same configuration system controls both experimentation and final reporting artifacts.

4 Dataset Overview

The dataset contains labeled training spectrograms and an unlabeled test split. The project uses both `accuracy` and `macro_f1` because the macro-F1 score provides a clearer view of per-class robustness than accuracy alone.

	metric	value
0	<code>num_train_images</code>	9227
1	<code>num_test_images</code>	3955
2	<code>num_classes</code>	4
3	<code>original_image_width</code>	51
4	<code>original_image_height</code>	45
5	<code>configured_input_size</code>	64
6	<code>configured_input_mode</code>	grayscale



5 Preprocessing Design

The final configuration uses a 64×64 grayscale input pipeline instead of the original larger RGB setup. This design was chosen for two reasons:

1. The original spectrogram images are very small, so aggressive upscaling to 224×224 increases computation without adding new semantic information.
2. The visible colors mainly represent a visualization colormap rather than true multi-channel sensor measurements, so grayscale is a reasonable and more efficient representation.

The preprocessing stage was therefore not only a computational optimization, but also a modeling choice. It reflects the assumption that the discriminative information of the spectrogram is mainly encoded in its structural intensity pattern rather than in arbitrary color-map variation.

5.1 Three Augmentation Strategies

The augmentation study in this project did not treat all image augmentations as equally suitable. Instead, three explicit strategies were designed and compared.

5.1.1 Strategy: none

- Purpose: Provide a clean reference point without any augmentation.
- Time mask: 0.00 / width 0
- Frequency mask: 0.00 / height 0
- Gaussian noise: 0.00 / std 0.0
- Amplitude scaling: 0.00 / [1.00, 1.00]
- Color jitter: 0.00

5.1.2 Strategy: light

- Purpose: Introduce mild perturbations while staying close to the original signal.
- Time mask: 0.15 / width 8
- Frequency mask: 0.15 / height 8
- Gaussian noise: 0.10 / std 0.01
- Amplitude scaling: 0.15 / [0.95, 1.05]
- Color jitter: 0.05

5.1.3 Strategy: spectral_safe

- Purpose: Apply stronger but spectrogram-consistent perturbations to improve robustness.
- Time mask: 0.35 / width 16
- Frequency mask: 0.35 / height 12
- Gaussian noise: 0.15 / std 0.015
- Amplitude scaling: 0.20 / [0.90, 1.10]
- Color jitter: 0.02

The logic behind the three strategies is the following:

1. **none**: This strategy disables all perturbations and serves as the pure reference condition. It is necessary because it shows how much performance the model can obtain from the original data alone.
2. **light**: This strategy introduces only small perturbations, including weak masking, weak Gaussian noise, and narrow amplitude scaling. The purpose is to improve generalization without significantly altering the visual structure of the spectrogram.

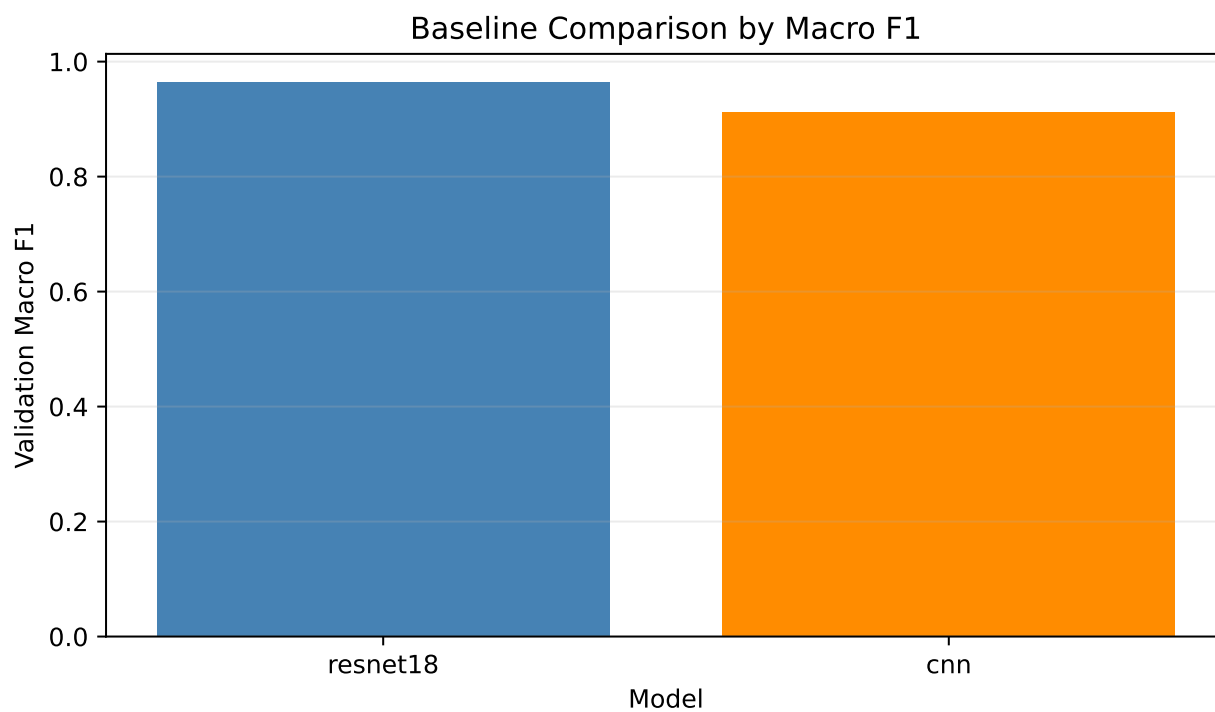
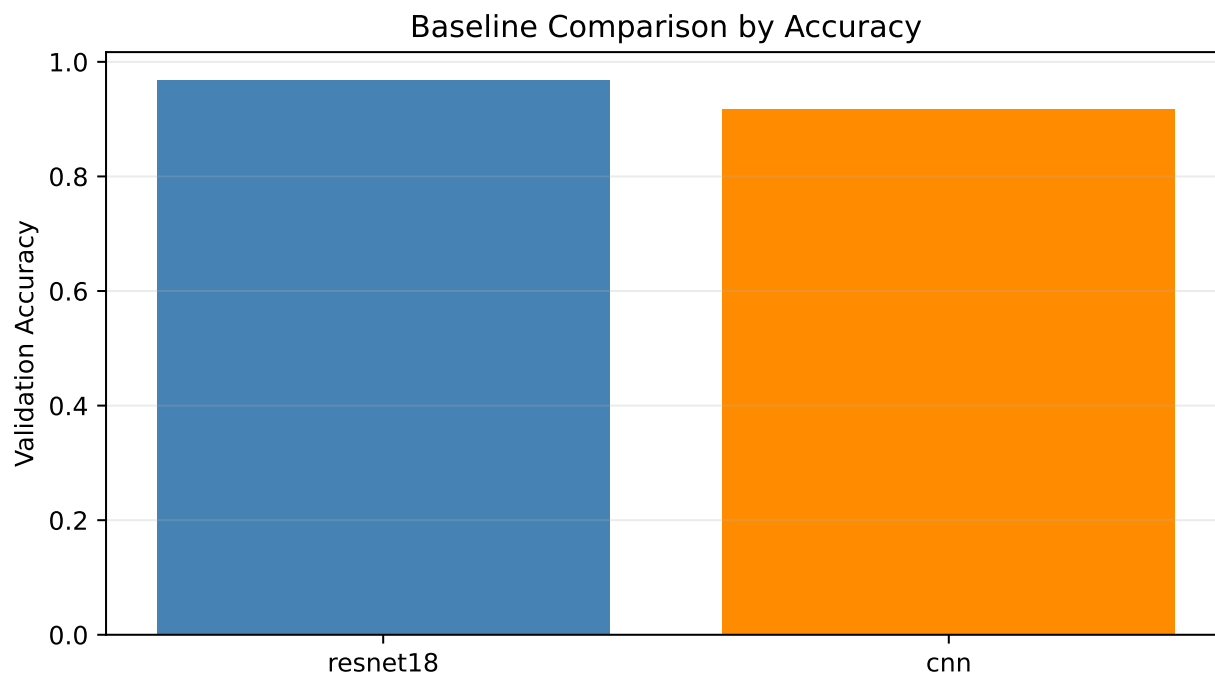
3. **spectral_safe**: This strategy applies stronger perturbations, but only in ways that remain consistent with spectrogram semantics. Instead of using unrealistic flips or strong color changes, it focuses on time masking, frequency masking, mild additive noise, and controlled amplitude scaling. This strategy was designed to increase robustness while preserving the temporal-frequency interpretation of the input.

In other words, the three strategies form a progression from no robustness intervention, to mild generic regularization, to a more domain-aware augmentation design.

6 Baseline Model Comparison

The first decision point in the project was whether a custom CNN baseline was sufficient or whether a stronger ImageNet-style backbone should replace it.

	model_name	valid_accuracy	valid_macro_f1	best_epoch	rank
0	resnet18	0.968581	0.965208	18	1
1	cnn	0.917660	0.911990	8	2



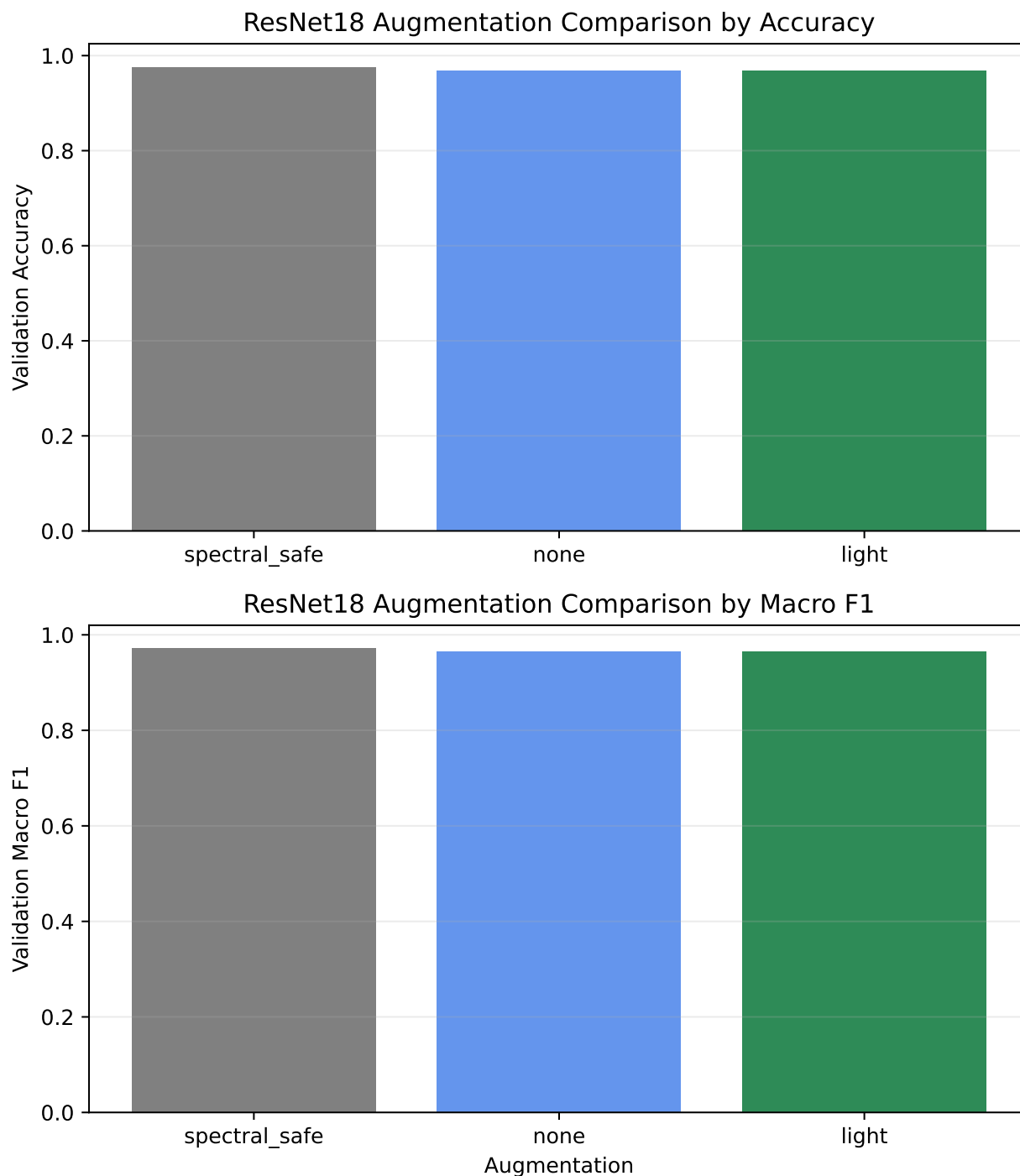
resnet18 clearly outperformed the custom **cnn**, so all later experiments focused on the **resnet18** family for the final system.

7 Augmentation Comparison

After fixing `resnet18` as the primary backbone, the next step was to compare augmentation presets.

This comparison is important because the performance gap between augmentation strategies explains not only which preset should be used in the final model, but also whether domain-aware perturbations are genuinely useful for this task.

	aug_name	valid_accuracy	valid_macro_f1	best_epoch	rank
0	spectral_safe	0.976165	0.971460	19	1
1	none	0.968581	0.965208	18	2
2	light	0.968581	0.964395	17	3



The comparison shows that **spectral_safe** is the strongest augmentation choice, outperforming both **none** and **light** on the holdout validation set.

This result is meaningful from a methodological perspective. The **light** strategy adds only limited regularization and therefore improves robustness less consistently. By contrast, **spectral_safe** applies perturbations that are stronger but still aligned with the time-frequency structure of spectrogram data. The experiment therefore suggests that the best augmentation policy is not simply the weakest one, but the one that introduces enough variation while respecting the semantics of

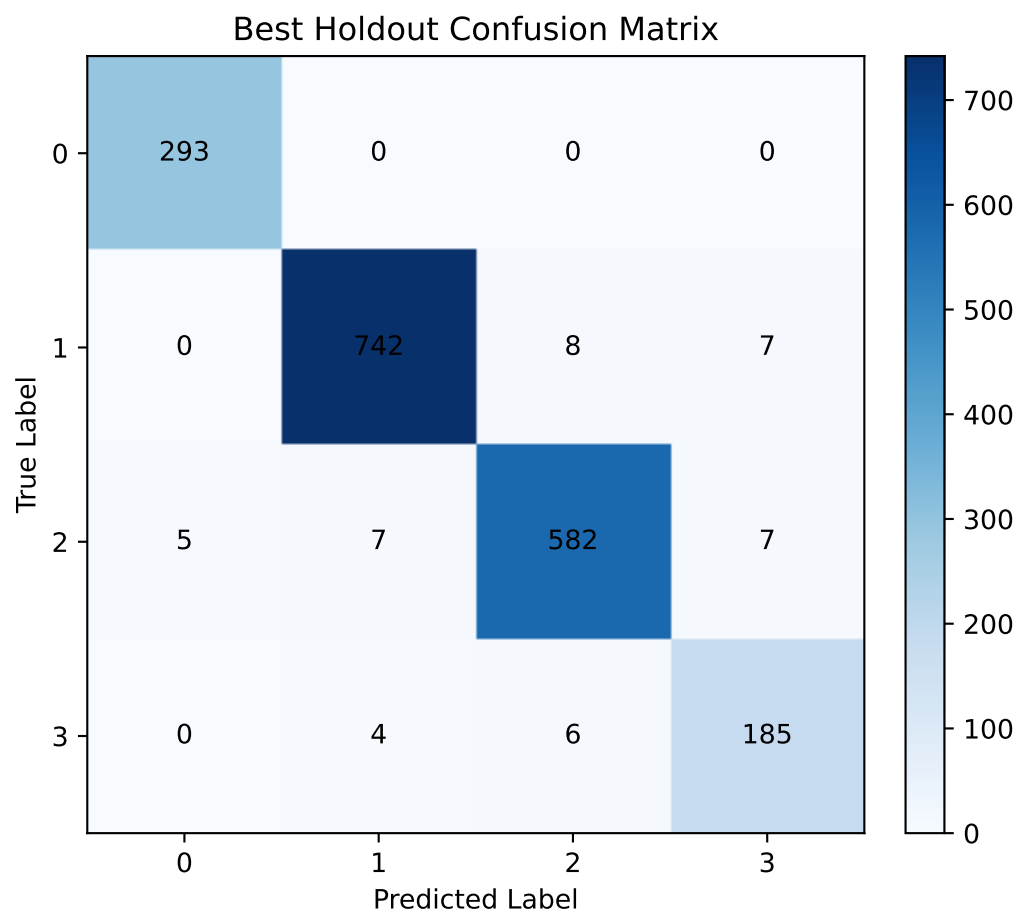
the representation.

8 Best Holdout Model Analysis

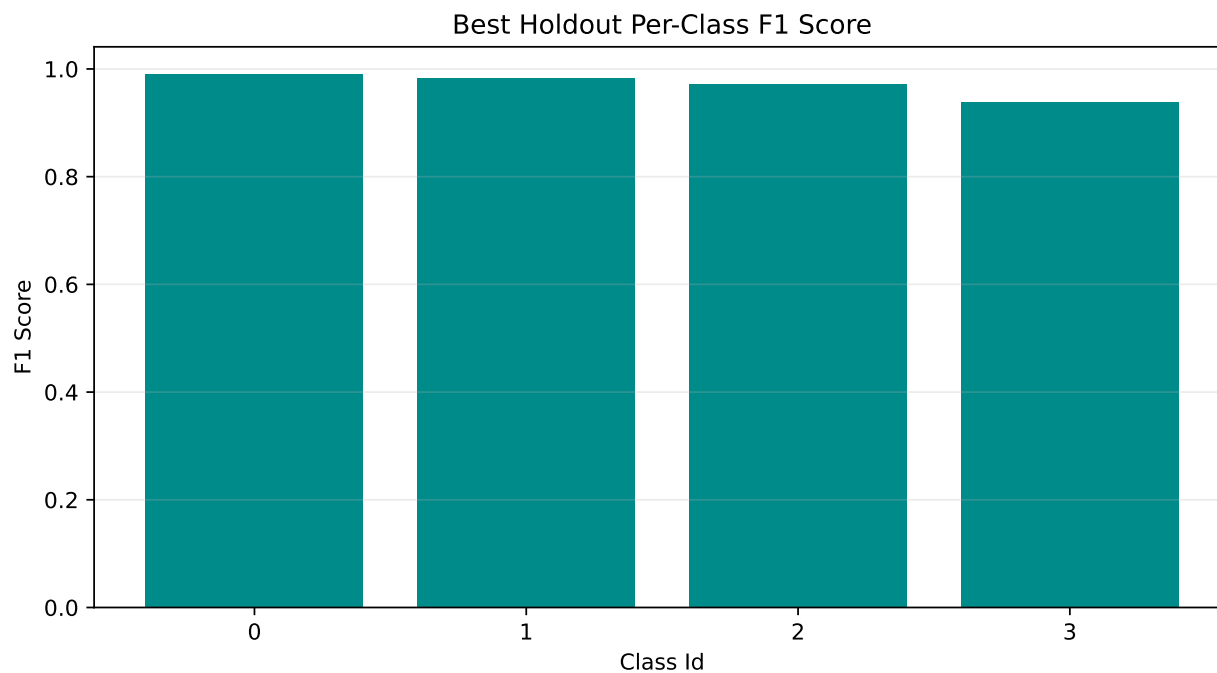
The best holdout run was obtained with `resnet18 + spectral_safe`.

8.1 Key Holdout Metrics

- Accuracy: 0.9761646803900325
- Macro F1: 0.9714596480443722
- Num Validation Samples: 1846
- Num Errors: 44
- Mean Prediction Confidence: 0.9790475688802073
- Mean Correct Confidence: 0.9837543394470321
- Mean Error Confidence: 0.7862839197570627
- Max Error Confidence: 0.997484564781189
- Most Common Confusion: 1->2
- Most Common Confusion Count: 8

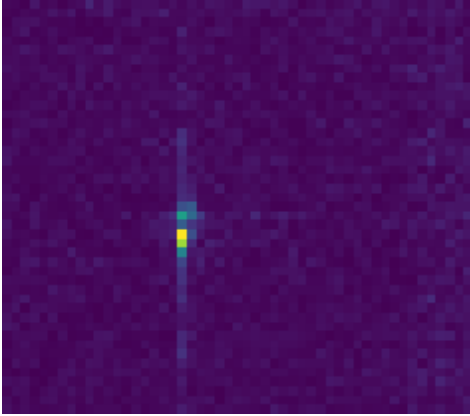


	class_id	support	predicted_count	correct_count	precision	recall	f1_score	target_count
0	0	293	298	293	0.983221	1.000000	0.991540	293
1	1	757	753	742	0.985392	0.980185	0.982781	757
2	2	601	596	582	0.976510	0.968386	0.972431	601
3	3	195	199	185	0.929648	0.948718	0.939086	195

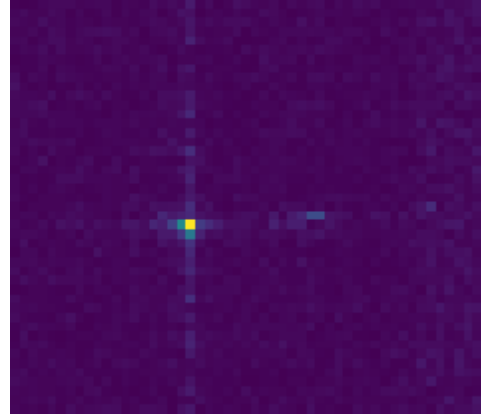


Top High-Confidence Misclassified Samples

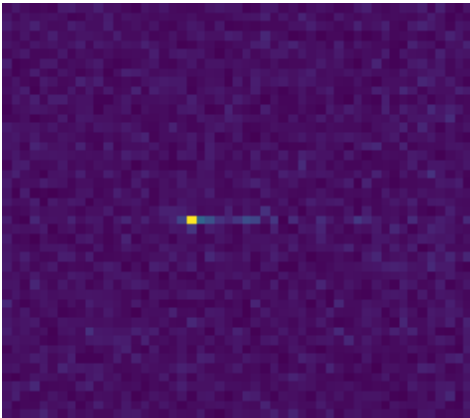
id=8905
true=1, pred=2
conf=0.997



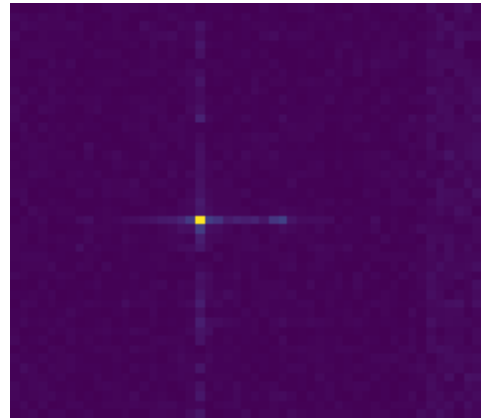
id=1822
true=2, pred=1
conf=0.995



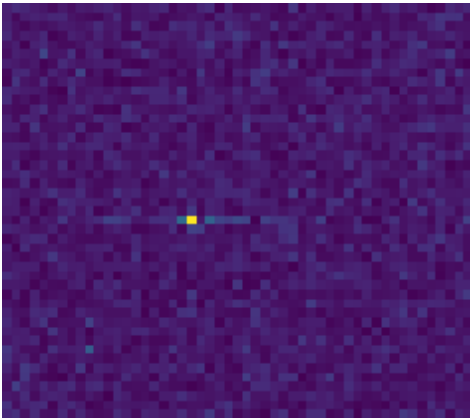
id=8180
true=2, pred=0
conf=0.994



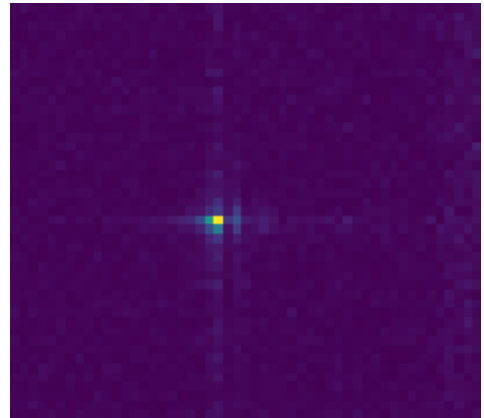
id=4342
true=1, pred=2
conf=0.985



id=2411
true=2, pred=0
conf=0.985



id=5525
true=2, pred=3
conf=0.983



The holdout analysis indicates that most remaining errors are concentrated in a small set of class confusions, especially around classes 1, 2, and 3. Class 3 remains the weakest class and is the main

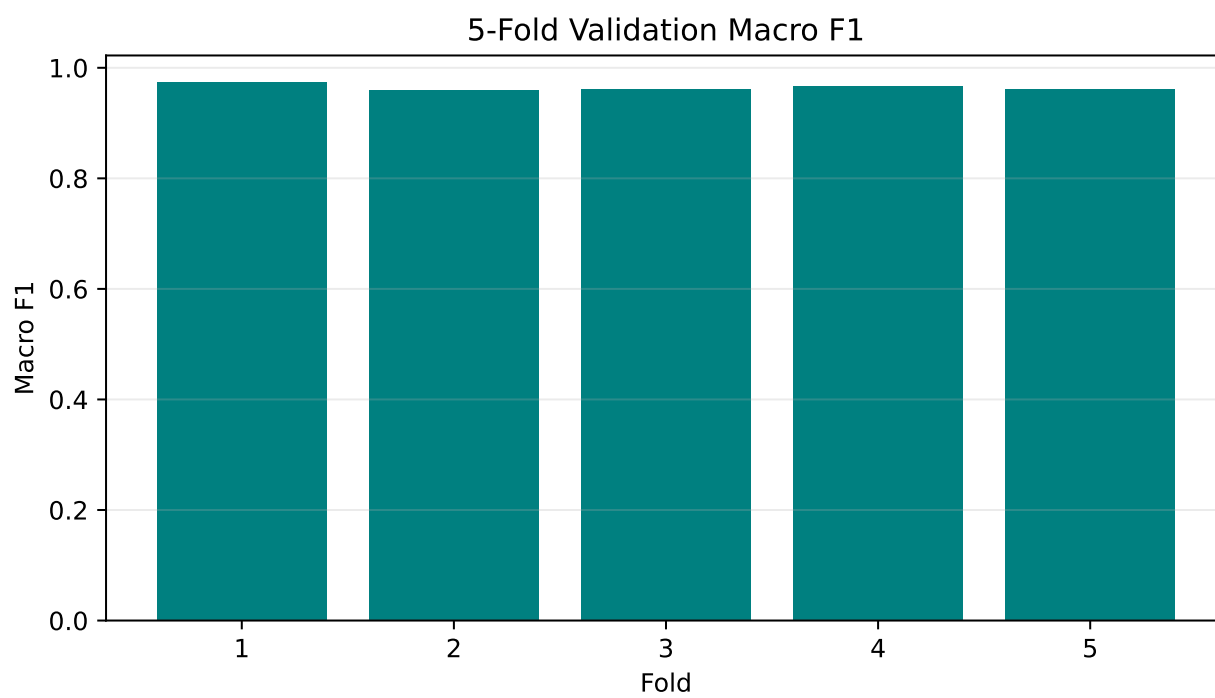
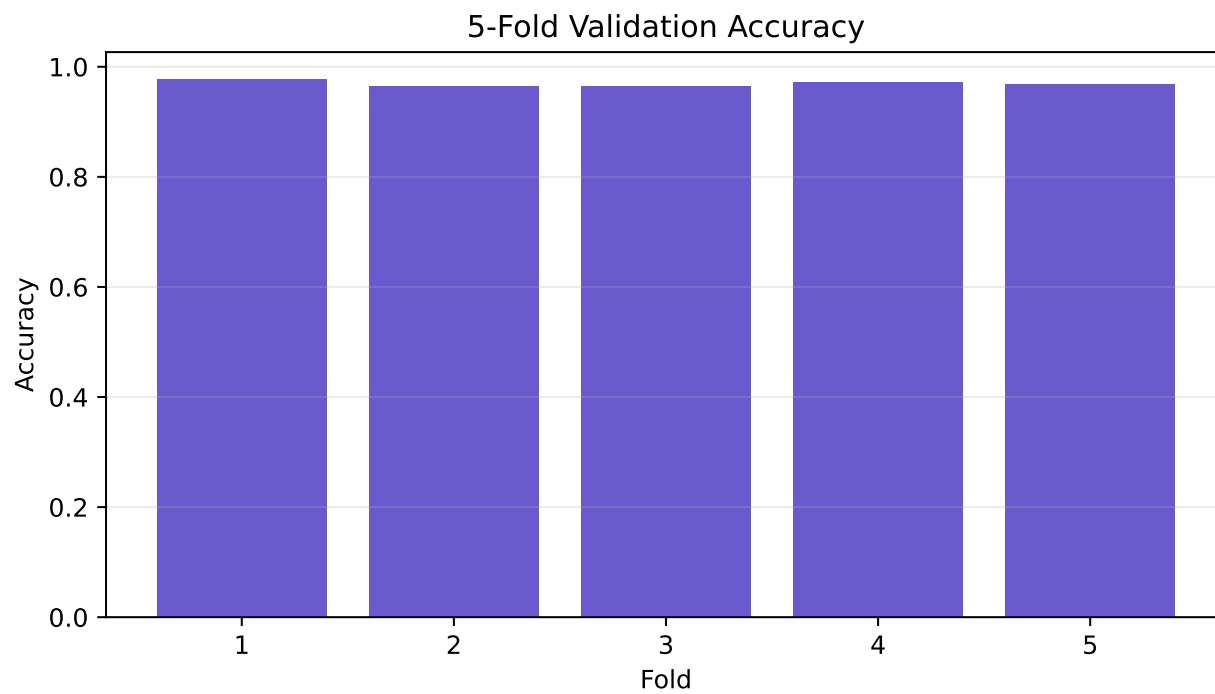
target for future improvement.

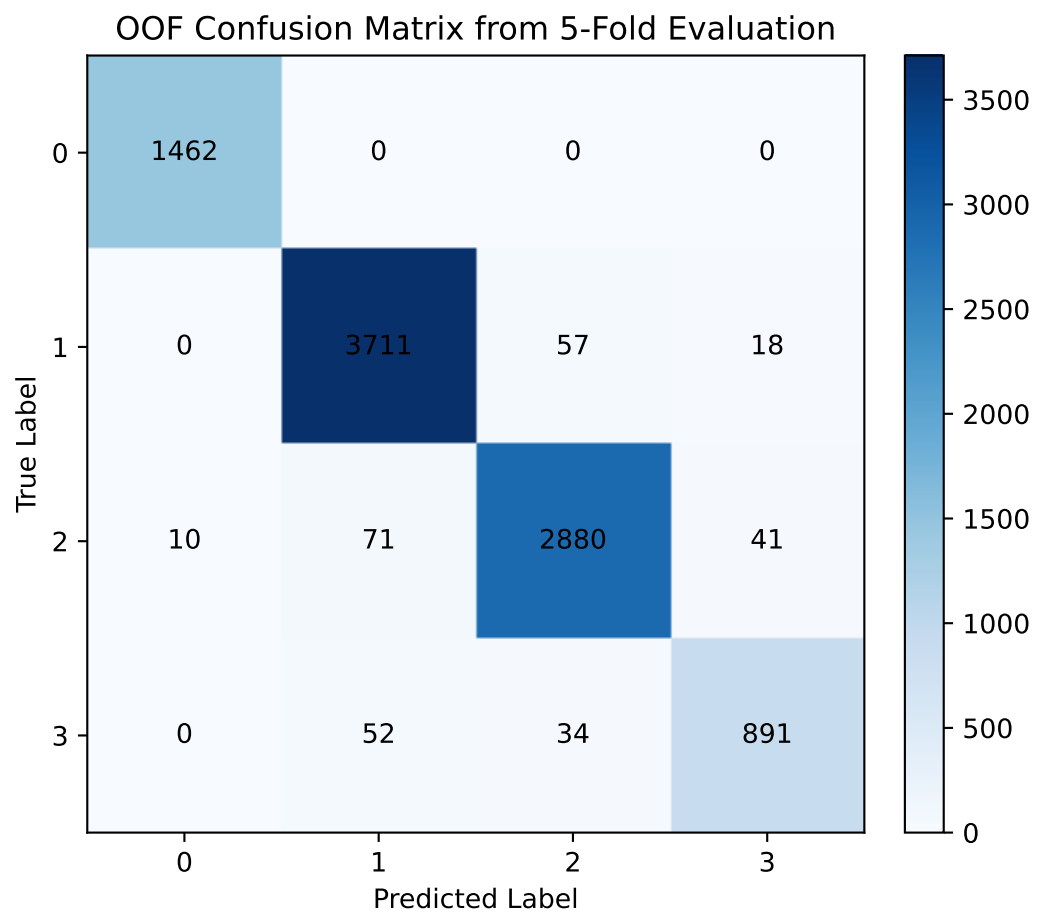
9 Final 5-Fold Cross-Validation Evaluation

To verify that the holdout findings were not accidental, the final configuration was evaluated with stratified 5-fold cross-validation.

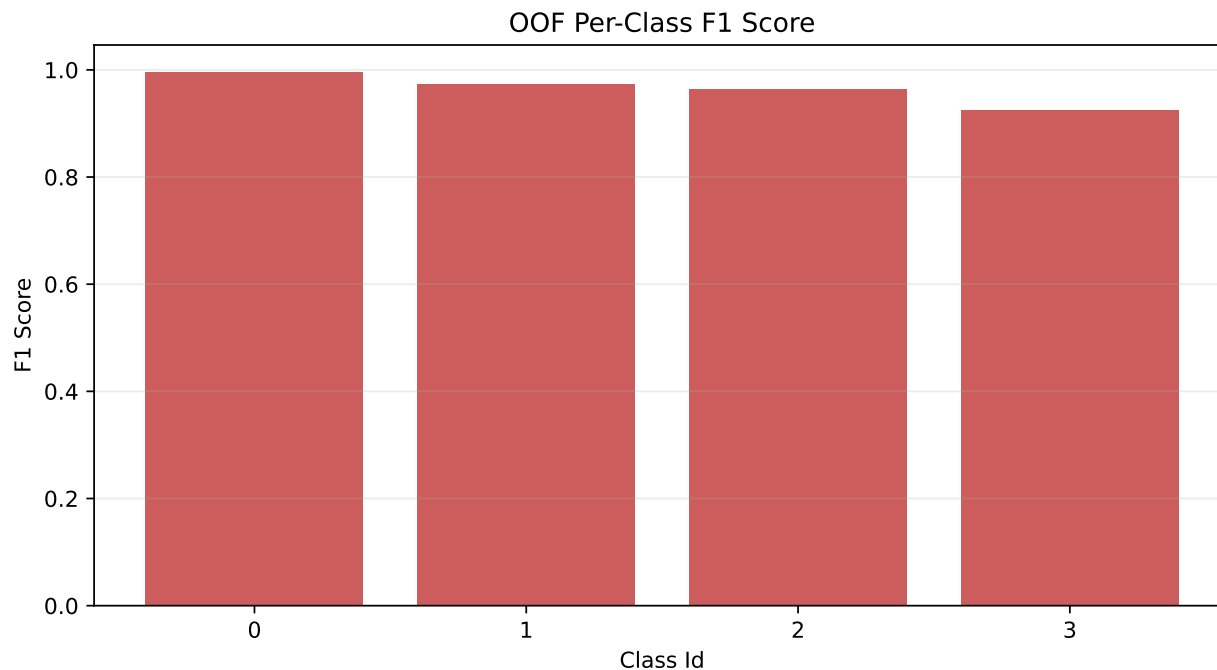
	metric	value
0	num_folds	5.000000
1	mean_valid_loss	0.115243
2	std_valid_loss	0.016778
3	mean_valid_accuracy	0.969329
4	std_valid_accuracy	0.005115
5	mean_valid_macro_f1	0.964951
6	std_valid_macro_f1	0.005103
7	best_fold_id	1.000000
8	best_fold_macro_f1	0.973623
9	best_fold_accuracy	0.977790

	fold_id	best_epoch	valid_loss	valid_accuracy	valid_macro_f1
0	1	20	0.091126	0.977790	0.973623
1	2	18	0.126242	0.964789	0.960004
2	3	19	0.139743	0.963686	0.961779
3	4	20	0.114011	0.971816	0.967862
4	5	17	0.105091	0.968564	0.961488





	class_id	support	precision	recall	f1_score
0	0	1462	0.993207	1.000000	0.996592
1	1	3786	0.967919	0.980190	0.974016
2	2	3002	0.969371	0.959360	0.964340
3	3	977	0.937895	0.911975	0.924754



The cross-validation results confirm the stability of the final configuration:

- Mean validation accuracy: 0.9693
- Mean validation macro F1: 0.9650
- Standard deviation of accuracy: 0.0051
- Standard deviation of macro F1: 0.0051

These values show that the holdout conclusion is not incidental. The final pipeline remains strong across multiple train/validation splits.

10 Final Submission Artifact

The current best submission was generated from the strongest holdout checkpoint:

	id	target
0	9227	2
1	9228	2
2	9229	0
3	9230	0
4	9231	1
5	9232	1
6	9233	1
7	9234	1
8	9235	1
9	9236	2

Submission path:

`datasets/submissions/resnet18/spectral_safe/submission_resnet18.csv`

11 Conclusion

This project established a complete, reproducible spectrogram classification workflow, including training, comparison, validation analysis, submission generation, and final 5-fold evaluation.

The main conclusions are:

- `resnet18` is a substantially stronger baseline than the custom `cnn`.
- Reducing the input to `64 x 64` and switching to grayscale improved efficiency without weakening the model.
- `spectral_safe` is the best augmentation preset among the tested options.
- The final configuration is both strong on the holdout split and stable under 5-fold cross-validation.

Future work can focus on deeper class-specific error analysis, stronger backbones, and more specialized spectrogram augmentations for the hardest classes.

Code repository: <https://github.com/cyang2001/Exercises>